

46-927, Spring 2021: Homework #4

Due 1:30PM EST, Wednesday, February 24, 2021.

Homework should be submitted electronically on Gradescope before the deadline. Please submit your homework as a single PDF file. We recommend using a Jupyter notebook to prepare your homework.

Note: For this homework, some problems are easier to do in R. (R's hierarchical clustering functions are easier to work with, but scikit-learn can also do hierarchical clustering if you prefer.) You can use one Jupyter Notebook, since Jupyter supports creating R code cells. You'll need to install the rpy2 package for Python (if it's not already included in your Anaconda setup) and, of course, R. Then, at the top of your notebook, include a cell that contains

```
%load_ext rpy2.ipython
```

and then start any cell containing R code with the line

```
%%R
```

Please post any questions on the Piazza discussion board.

1. *Choosing the number of clusters in K-means.* In class, we discussed the K -means algorithm. Here we will see an example where K -means can be applied, and explore how to choose K appropriately.

For this problem, we will use the dataset described here: <http://rosmarus.refsmmat.com/datasets/datasets/nutrition/>

(You do **not** need to answer the questions listed on that page for this problem, only the questions listed below.)

Read the CSV file into a Pandas data frame. Review the website's description of the data to ensure you're familiar with the variables it includes.

- (a) We'd like to see if clustering food by nutrition produces interesting and meaningful clusters. To do that, subset the columns of the data frame to select the following variables: `Energ_Kcal`, `Protein`, `Lipid_Tot`, `Carbohydrt`, `Fiber_TD`,

`Sugar_Tot`, `Cholestl`. Use the `dropna` method on the resulting data frame to remove any rows containing NaN values (which represent nutrition data that wasn't measured), then convert to a Numpy array with `to_numpy()`. Use the `sklearn.preprocessing.scale` method to scale the data matrix so each column has standard deviation 1; this avoids problems caused by differences in scales between columns.

`sklearn.cluster.KMeans` does K -means clustering, and its helpful `fit_predict` method returns the cluster index ($C(i)$, in the notation of lecture) for each row of data.

Run a K -means clustering with $K = 5$ on the whole dataset and store the cluster indices.

Row indices 3 through 39 are all varieties of cheese (such as `CHEESE,SWISS` and `CHEESE,PAST PROCESS,AMERICAN,FORT W/ VITAMIN D`). Look at the clusters the cheeses fall into. Are they clustered together? If not, does the split between clusters make sense?

- (b) We picked $K = 5$, but given how many types of food there are, it's far from obvious that there should only be 5 clusters. We have no true underlying cluster labels, so the "right" K depends on our goals: if the Department of Agriculture has 50 inspectors and wants each inspector to specialize in similar foods, we can just set $K = 50$ and be done.

But if we're interested in finding some "natural" grouping of the data that seems to represent some real feature of the data, we need a better rule. One way is to examine the within-cluster variation W . Let W_K be the within-cluster variation we obtain for a particular K .

First: argue (in a sentence or two) that the within-cluster variation **must** decrease when we increase K .

- (c) Suppose the data truly does fall into K^* distinct groups. We would expect that for $K \ll K^*$, $W_{K+1} < W_K$, and the gap will be quite large. When $K > K^*$, the inequality is still true, but the gap will be smaller.

Make a plot of W_K for K from 5 to 100. (The object returned by `KMeans.fit` has an `inertia_` attribute, which is scikit-learn's name for W .) Is there a "kink" in the plot where W starts to decrease more slowly?

(This is only a heuristic, not a strong rule, so it may be hard to identify an exact point where this happens.)

- (d) Identifying a kink in a graph perhaps does not seem rigorous enough. Another approach uses the *gap statistic*, which compares W_K with the value we'd obtain

using the same clustering algorithm on null data—data with no cluster structure at all.

For each value of K , we calculate:

- $\log W_K$, using K -means on our dataset.
- $\log W_K^*$, using K -means on a null dataset of the same size as the real data. We make a null dataset by drawing uniformly at random over a rectangle in $\mathbb{R}^p$ that contains the real dataset. We repeat this B times, say $B = 20$, and take the average of $\log W_K^*$ over the B simulations.

We can then calculate the gap statistic

$$\text{gap}_K = \log W_K^* - \log W_K.$$

When the cluster structure is strong, plotting k versus gap_k will reveal a local maximum: the “best” k for this dataset. (Here “best” means it produces clusters packed the best compared to the clustering we’d get if there was no cluster structure in the data at all.) In more difficult cases, like ours, this won’t work—the gap statistic appears to increase indefinitely. We need a cutoff to tell when gap_k has stopped growing meaningfully.

We calculate a new quantity sd_k , the sample standard deviation of the within-cluster variation the B random datasets drawn for a given k . Let $s_k = \text{sd}_k \sqrt{1 + 1/B}$.

To estimate the best K , we can choose

$$\hat{k} = \text{smallest } k \text{ such that } \text{gap}_k \geq \text{gap}_{k+1} - s_{k+1}.$$

Write code to run this procedure for the nutrition data, and apply it specifically for K from 10 to 30. Does the $\hat{k}$ you find match what you chose in part (c)?

(You can use `np.random.uniform` to draw data uniformly at random over rectangles. The `size` argument lets you pick the size of the generated array, so you can make it match the original dataset. `np.min` and `np.max` can be used to pick lower and upper bounds forming a rectangle around the real data.)

If you’re interested, you can read more about the gap statistic and its motivation at <https://statweb.stanford.edu/~gwalther/gap>

2. *Clustering the Dow Jones Industrial Average.* You will cluster the daily returns for the Dow Jones Industrial Average (DJIA) stocks to see if we detect basic patterns. In doing so, you’ll get some experience with distance metrics and clustering methods.

The `djia_data.RData` file contains a data frame of daily closing prices for the DJIA. You can load this file with the `load` function in R. Note that the first 30 columns are stock data, while the last column is dates.

The `djia_info.RData` file contains the descriptions of DJIA stocks from the Wikipedia page. In particular, it maps the symbol to the stocks and to categorizations of their industries. We'll use these categories to sanity check our clusters. You can also load this file with the `load` command.

Note: The R code we used in class to run `hclust` is available on Canvas, under Files for Lecture 4.

- (a) Load the data set. Compute the daily log returns from this sequence. Make a plot with all the log returns. This should look like 30 time series lines on the same plot. Make sure to set the vertical axis limits so all the data fit (you can set the `ylim` parameter in `plot`).
- (b) Use the `dist` command to compute a distance matrix for the log returns. Right now, you'll just be using Euclidean distance (the default). Call `hclust` on the distance matrix with the `complete` method and plot the results. Decide a reasonable number of clusters based on your tree and give a sentence or two of justification. Do you see anything weird in your tree?

Hint:

The `dist` function acts on matrices, so you need to convert the columns of interest to a matrix with `as.matrix`. The `dist` function also computes distance between rows, not columns, so you need to transpose your matrix. The result would be something like `t(as.matrix(your_data_frame_columns))`

(Yes, I know that there's a reason for the something weird.)

- (c) Use the `cutree` command to select 3 groups. What industries are represented in each group? Can you make up a general story for the division you're seeing?

Hints:

The `cutree` command produces a vector of group numbers (1, 2, or 3), assigning each stock to one of three clusters. Each entry also has a name, which is the stock symbol. The helper function `resolve_groups` maps the group number vector to groups of symbols and groups of industries. See the comments in `djia_helpers.R` for usage.

- (d) Use the same distance matrix and `hclust`, this time with the `single` method instead. Plot the tree. Does it look as nice as the complete linkage tree?
- (e) The results in Parts (b) and (c) were a little strange, because our distance measure was just the Euclidean distance between log return vectors. You saw in (a) that the returns can be on quite different scales. You might instead want to use correlation, which will normalize the vectors.

We can construct such a distance matrix by computing a matrix of $1 - \text{correlation}$ (since we want high correlations to give small distances). We use the `as.dist` function to change this matrix into a distance object that `hclust` understands. Construct such a correlation-based distance based on your log returns, and rerun your complete linkage clustering. Plot the tree, extract 4 groups, and report the stocks and categories. Can you see any reasonable pattern?

Hints:

To make it more explicit, `data` were a set of columns of interest, `cor(data)` would compute a correlation matrix, so `1 - cor(data)` would be a reasonable measure of distance. Then `as.dist(1 - cor(data))` would give an actual distance object for `hclust`

You might be bothered by the fact that we're not being careful with negative correlations. Luckily there are none in this data set.

- (f) Finally, you might be interested in finding non-linear relationships between your stocks. *Spearman correlation* is one such measure. This is a correlation-type measure that is high if variables could be monotonically transformed to be correlated. As such, it's invariant to monotone transformations of the variables, and much more robust to outliers.

Change your code from Part (e) to cluster based on Spearman correlation. All you have to do is add `method='spearman'` to your `cor` command, and nothing else should change. Plot the cluster extract 4 groups, and figure out the stocks and categories.

Do you notice anything different? In particular, what happens to NKE? Why do you think this is?

Just for fun (ungraded and not turned in): You should check out Spearman correlation, because it's quite cool. I recommend making one vector of random x values, and one vector of x^2 values (for positive x). If you compute their correlation, it won't be very high. If you compute their Spearman correlation, it should be 1. (The `cor` function takes a `method='spearman'` argument.)

3. In this problem, we will explore a couple different clustering methods and distance measures on a data set of the handwritten digits 0, 1, and 4. The file `x_014_tr.csv` contains images of handwritten digits, one image per row (unwrapped into a single row), with the columns representing individual pixel values. (Each pixel is a single number, representing its grayscale value.) The file `y_014_tr.csv` contains the digit labels. The function in `plot.digit.R` reformats a row as an image and plots it for you.

Load the data set and see what is in it.

- (a) Create a distance matrix using Euclidean distance on your digits (just call the `dist` function). Perform hierarchical clustering with complete linkage clustering. Plot this tree. Compare the complete linkage clustering with 3 groups to the true digit identities in `y_014_tr` (use `table` or `xtabs`). How well do they match? To see if it's just the wrong number of clusters, also produce the clustering with 6 groups. Do these do a better job of separating the digits? (Since we have more clusters than digits, a good job would mean that each cluster corresponds mostly to one digit, though each digit can show up in multiple clusters.)
- (b) We will now try changing distances. Load `tangent_distances.RData`, which will add a variable `tangentDist` to your workspace. This contains a clever version of distance that is designed to behave a bit better for handwritten digits. This is a distance object that you can feed into hierarchical clustering. (I give you this precomputed form because the computation of these distances takes a while.)
- Try complete linkage clustering again using this distance, plot the tree and compare your results for 3 and 6 clusters.
- (c) For one last clustering, try prototype clustering on the `tangentDist` distances. ("Prototype clustering" is another name for hierarchical clustering with the min-max linkage. It has this name because it produces "prototypes" for each cluster: a single representative data point for the cluster.) Plot the tree and compare the 3 and 6 cluster performance. (Focus especially on the performance at separating 1s from 4s, the hardest task in previous problems.)

To run prototype clustering, you need to use the `protoclust` library instead of `hclust`. The function inside this library is also called `protoclust`, and the function to cut the tree is `protocut`. Along with producing a cut, the `protocut` function also returns the group representative indices in `protos`. The syntax is very similar to `hclust`, but take a quick look at the help file.

- (d) Let's take complete and prototype clustering with 6 clusters. In reality, you don't know what digits these might correspond to. Suppose that you wanted to summarize the clusters from each method. Produce a 2×3 plot for each method, showing the following summary of each cluster:
- For complete linkage clustering, average the images corresponding to each digit assigned to that cluster. Plot that average image.
 - For prototype/minimax clustering, plot the representative of each cluster.
- Which one is easier to understand?